

SafeSlice: Safe Dynamic Network Slicing for SLA- Preserving SDN

A lightweight contextual-bandit controller with a deterministic safety guardrail,
evaluated in a queue simulator and on a real Mininet / Open vSwitch testbed

PROJECT REPORT

Course	Computer Networks (project component)
Team members	Prithvi Raghu (24BCE2624) Sumanta Kumar (24BCT0302)
Guide	Sasikala R , SCOPE, VIT
Domain	Computer Networks · Software-Defined Networking · Machine Learning
Repository branch	<code>feature/mininet-testbed</code>

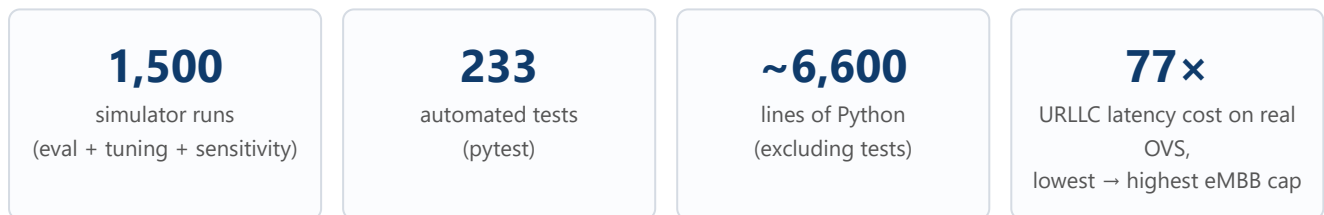
Abstract

Three kinds of traffic share one bottleneck link and want incompatible things: **URLLC** needs low latency, **eMBB** wants as much throughput as possible, and **Best Effort** takes what is left. A fixed bandwidth split wastes capacity, or else lets an eMBB burst starve URLLC. Most published work on dynamic slicing uses deep reinforcement learning. That needs long training and a lot of compute, and nothing limits what the policy does while it is still learning.

SafeSlice tests a lighter design. Once per second a **LinUCB contextual bandit** reads a 9-feature context and picks one of five absolute rate caps for the eMBB slice. A **deterministic safety guardrail** sits between the policy and the network. It masks aggressive choices when URLLC delay passes 4 ms, forces the safest level when it passes 7 ms, and then holds that level for a cool-down period.

The project was built in two phases. In **Phase 1** we wrote a token-bucket queue simulator and ran a full policy study: 320 evaluation runs, 280 tuning runs and 900 reward-sensitivity runs, 8 policies × 4 traffic scenarios × 10 seeds, with paired statistics. In **Phase 2** we built a real **Mininet + Open vSwitch** testbed on WSL2 in five gated stages and used it to check the simulator's key modelling assumption.

Main findings: (1) online LinUCB *loses* to a well-tuned threshold rule on 3 of 4 scenarios, but the same algorithm pre-trained and frozen *wins* on 3 of 4 with fewer SLA violations, so the cost of exploration decides the outcome; (2) a context-free bandit is caught by traffic spikes more than twice as often, so the context is what the bandit adds; (3) the guardrail provably keeps every action inside the allowed set, but it cannot prevent every violation; (4) on real OVS, raising the eMBB cap costs URLLC about 77× in median latency, so the control problem is real, and the simulator's sharing mode is the one that matches the switch.



Keywords: network slicing, SDN, contextual bandit, LinUCB, safe reinforcement learning, Open vSwitch, HTB, Mininet, URLLC, eMBB.

Contents

1. Introduction	6. Implementation and Testing
1.1 Motivation · 1.2 Problem statement · 1.3 Objectives · 1.4 Scope	7. Experimental Methodology
2. Literature Survey	8. Results: Simulator Policy Study
3. System Overview and Architecture	9. Real Testbed: Mininet and Open vSwitch
3.1 Three slices · 3.2 Architecture · 3.3 Control loop	10. Simulator vs Real Switch
4. Module Design	11. Discussion and Key Findings
Configuration · Traffic · Backend interface · Simulator · Context · Reward · Policies · LinUCB · Guardrail · Runner	12. Limitations
5. Algorithms	13. Conclusion and Future Work
5.1 LinUCB · 5.2 Guardrail · 5.3 Token-bucket sharing	References
	Appendix A. Reproducing the results
	Appendix B. Repository map

List of Figures

Figure	Title
1	Static vs dynamic slicing: the problem
2	SafeSlice system architecture
3	One control step (1 second), sequence view
4	Token-bucket queue simulator
5	Guardrail decision flowchart
6	Offered load of the four traffic scenarios
7	Headline: paired reward difference vs threshold
8	Safety / throughput trade-off per scenario
9	Guardrail interventions and escalation signals
10	Reward over time on the adversarial scenario
11	Time series: pre-trained LinUCB on adversarial
12	Project timeline and testbed stages
13	Testbed topology on Mininet / OVS
14	Real OVS level sweep vs simulator

1. Introduction

1.1 Motivation

5G-style networks carry very different services over shared infrastructure. **Network slicing** divides a physical link into logical slices, each with its own performance targets. Software-Defined Networking (SDN) makes slices *programmable*: a controller can change each slice's rate limit while the network is running. The open question is *how* the controller should decide, second by second, without breaking the latency promise (the SLA) of the most sensitive slice.

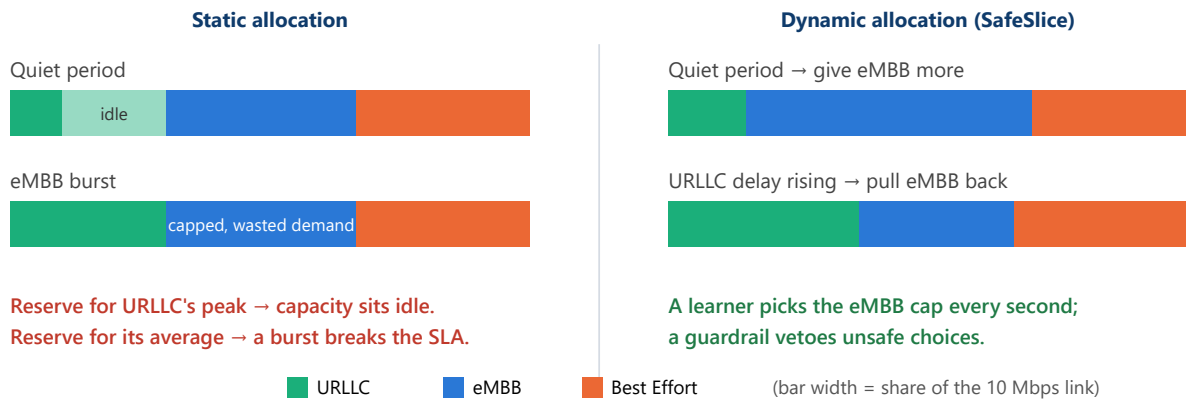


Figure 1. Why static slicing fails and what dynamic slicing changes.

1.2 Problem statement

Build a lightweight, AI-driven slicing controller that adapts bandwidth in real time while protecting URLLC latency, and state precisely what its safety mechanism does and does not guarantee.

The research question: *can a contextual bandit plus a deterministic safety mechanism beat a well-tuned reactive rule?* We accepted from the start that the answer might be "no". A clean negative result obtained with a sound method was treated as better than a forced win.

1.3 Objectives

#	Objective	Status
O1	Model a 10 Mbps bottleneck shared by URLLC, eMBB and Best Effort, with rate-capped queues that behave like OVS HTB queues.	✓ Done
O2	Design a 9-feature context, a reward function and a 5-level absolute action space.	✓ Done
O3	Implement LinUCB, an ϵ -greedy ablation, a threshold rule, static baselines and an oracle, all behind one interface.	✓ Done
O4	Implement a deterministic guardrail with a provable guarantee on the applied action.	✓ Done
O5	Run a statistically fair comparison: paired seeds, held-out tuning seeds, confidence intervals, reward-weight sensitivity.	✓ Done
O6	Build a real Mininet / OVS testbed and validate the simulator's key assumption against it.	✓ Done (one operating point)
O7	Run the learned policies live on the real switch.	✗ Future work

1.4 Scope

The project covers a single bottleneck link, three slices and one control interval (1 s). Only the eMBB cap is controlled. URLLC is never capped, and Best Effort has a fixed cap. The "SDN" data plane is real Open vSwitch with HTB QoS queues, configured through the OVS management interface rather than an OpenFlow controller (Section 12).

2. Literature Survey

#	Work	Approach	Gap SafeSlice addresses
[1]	Koo et al., <i>Deep Reinforcement Learning for Network Slicing</i> , 2019	DRL agent allocates slice resources from observed demand.	Needs heavy training; no bound on actions taken while learning.
[2]	Wei et al., <i>Dynamic Network Slice Reconfiguration using DRL</i> , 2020	DRL-driven reconfiguration of slices over time.	Reconfiguration cost and exploration risk are not isolated.
[3]	Dayot et al., <i>Deep Contextual Bandit-Based Slice Provisioning</i> , 2022	Contextual bandits for provisioning decisions.	Closest to our work; no deterministic safety layer and no online-vs-converged split.
[4]	<i>Safe and Fast RL for Network Slicing Resource Allocation</i> , 2023	Adds safety constraints to RL-based slicing.	We use a model-independent guardrail and state exactly what it guarantees.
[5]	Xie et al., <i>DRL-Based Resource Allocation for Network Slicing</i> , 2022	DRL for multi-slice resource allocation.	Compute cost; we show a 9×9 linear model is enough at this scale.
[6]	Li et al., <i>A Contextual-Bandit Approach to Personalized News Article Recommendation (LinUCB)</i> , WWW 2010	Disjoint linear UCB for contextual bandits.	The algorithm we adopt, applied here to network control.

No DRL baseline was implemented. References [1], [2] and [5] are cited for motivation and are **not** compared against by measurement.

Where SafeSlice sits

Property	Static split	Threshold rule	Deep RL	SafeSlice (LinUCB + guardrail)
Adapts to demand	No	Yes	Yes	Yes
Uses multi-signal context	No	No (1 signal)	Yes	Yes (9 features)
Training cost	None	Hand-tuning	High	Low (seconds)
Decision cost	~0.02 ms	~0.04 ms	NN forward pass	~0.09 ms
Bounded actions while learning	n/a	n/a	No	Yes (guardrail)

3. System Overview and Architecture

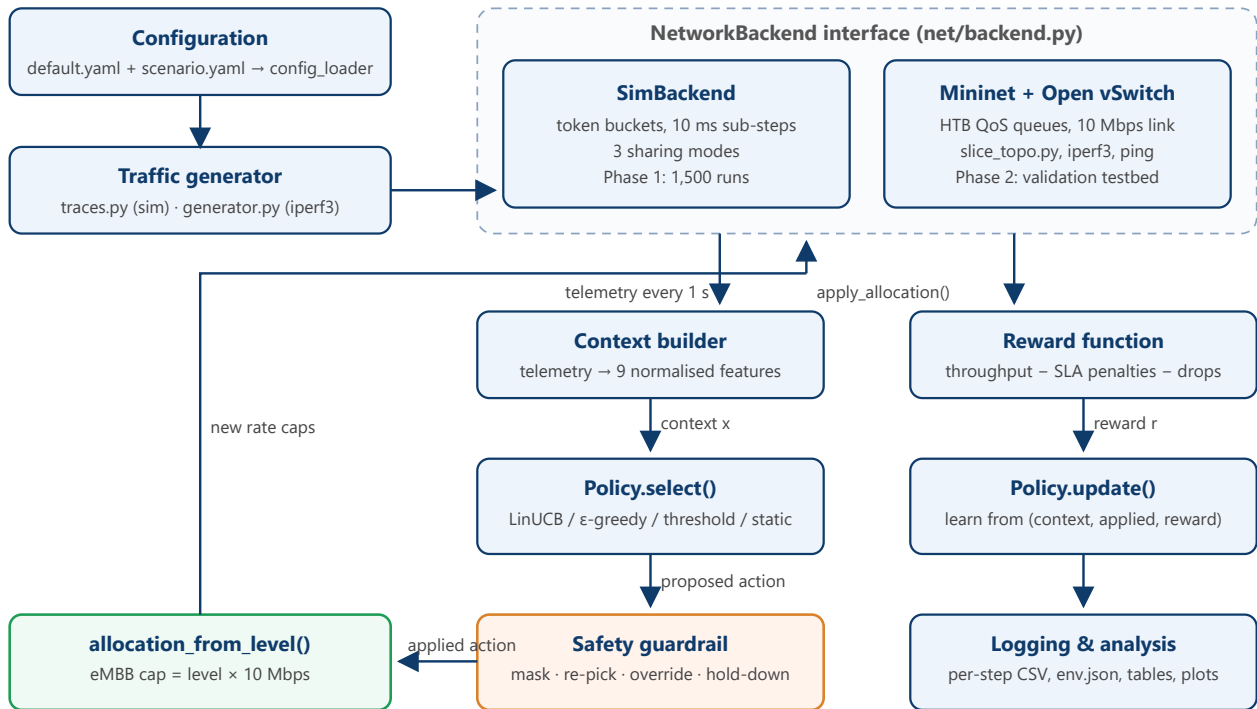
3.1 The three slices

Queue	Slice	Requirement	Guaranteed min	Cap
q0	URLLC	Low latency. The protected slice.	10 %	Never capped
q1	eMBB	Maximum throughput. The slice we control.	10 %	One of 20 / 35 / 50 / 65 / 80 %
q2	Best Effort	Whatever is left.	5 %	Fixed at 45 %

Why absolute levels rather than "increase / hold / decrease"? With relative actions, the effect of an action depends on the current allocation. The allocation then becomes part of the state, the problem becomes a full Markov decision process, and a bandit is the wrong tool. An absolute level means the same thing every time, so the bandit's assumption holds approximately.

Why is Best Effort's cap not tied to the action? If BE's cap shrank exactly as eMBB's grew, total demand would stay constant and the action would have no effect on URLLC. With a fixed BE cap, raising eMBB really does take capacity away from URLLC.

3.2 Architecture



Driven once per second by experiments/run_experiment.py (run_once). Policies, guardrail and metrics never know which backend is used.

Figure 2. SafeSlice architecture. The dashed box is the backend abstraction: the same agent code drives either the simulator or the real switch.

3.3 The control loop

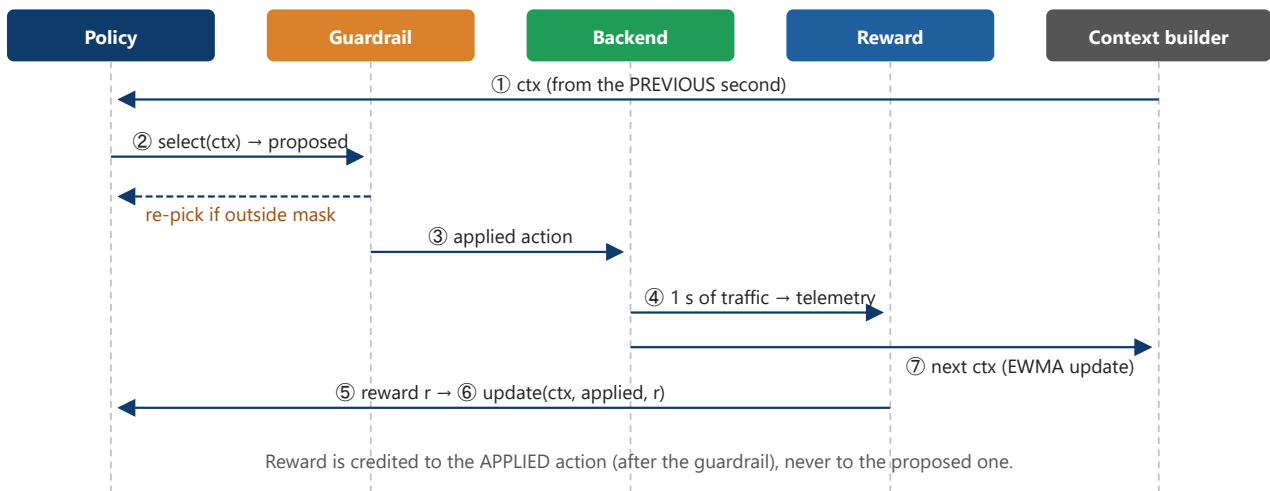


Figure 3. One control step. Decisions use only the previous second's measurements, as a real controller would.

```

ctx = ctx_builder.update(backend.idle_sample())
for step in range(n_steps):
    proposed = policy.select(ctx, unrestricted)
    decision = guardrail.apply(ctx, proposed, policy)
    backend.apply_allocation(allocation_from_level(cfg, decision.applied_action))
    tel = backend.read_telemetry()
    rb = reward_breakdown(tel, cfg)
    policy.update(ctx, decision.applied_action, rb.total)
    rows.append({...})
    ctx = ctx_builder.update(tel)

```

- # starting context
- # one loop = one second
- # 1. policy proposes
- # 2. guardrail checks
- # 3-4. 1 s passes, measure
- # 5. score it
- # 6. learn
- # 7. log everything
- # 8. context for next second

4. Module Design

#	Module	Files	Responsibility
1	Configuration	<code>config/default.yaml</code> , <code>config/scenarios/*.yaml</code> , <code>config_loader.py</code>	Every number in one place; read-only, typo-safe, hashable config.
2	Traffic	<code>traffic/traces.py</code> , <code>traffic/generator.py</code>	Reproducible offered load (sim); iperf3 / ping orchestration (testbed).
3	Network interface	<code>net/backend.py</code>	Abstract <code>NetworkBackend</code> , <code>TelemetrySample</code> , <code>Allocation</code> .
4	Simulator	<code>net/sim_backend.py</code>	Token-bucket queues on a shared 10 Mbps link.
5	Testbed topology	<code>net/topology/slice_topo.py</code>	Mininet topology, OVS HTB queues, flows, self-checks.
6	Context	<code>agent/context.py</code>	Telemetry → 9 normalised features.
7	Reward	<code>agent/reward.py</code>	Per-second score with an itemised breakdown.
8	Policies	<code>agent/policies/*.py</code>	Static, threshold, bandit adapters, oracle.
9	Bandits	<code>agent/bandit.py</code>	<code>LinUCB</code> and <code>EpsilonGreedy</code> .
10	Guardrail	<code>agent/guardrail.py</code>	Deterministic safety layer.
11	Experiments	<code>experiments/*.py</code>	Single run, suite, tuning, sensitivity, 5 testbed experiments.
12	Analysis	<code>analysis/*.py</code>	CSV schema, metrics, aggregation, figures.

4.1 Configuration

All parameters live in `config/default.yaml` . A scenario file is deep-merged on top of it. `load_config()` returns a read-only `cfg` object that allows attribute access (`cfg.link.capacity_bps`) and raises an error showing the full key path on a typo. `--set key=value` overrides work from the command line, and `config_hash()` stamps every run with a fingerprint of the settings used.

Setting	Value	Meaning
link.capacity_bps	10,000,000	Shared bottleneck, 10 Mbps
run.duration_s / warmup_s	300 / 30	Run length; first 30 s ignored
run.control_interval_s	1.0	One decision per second
action.embb_levels	[0.20, 0.35, 0.50, 0.65, 0.80]	The five eMBB caps
sla.target_ms / hard_ms	5.0 / 7.0	URLLC delay target and hard limit
guardrail.warn_ms	4.0	Start masking aggressive levels
guardrail.holddown_steps	3	Cool-down after an emergency
policy.linucb.alpha	0.5	Exploration strength
reward.w_tput, w_be, w_sla, w_viol, w_drop	1.0, 0.30, 1.0, 2.0, 0.50	Reward weights

4.2 Traffic generator

Scenario files describe **phases** (duration and offered Mbps per slice, optionally ramping). `build_trace()` expands them into a 10 ms schedule and adds mean-preserving log-normal jitter. Seeds are derived with **blake2b** rather than Python's `hash()`, which is salted per process, so the same (scenario, seed) always produces bit-identical traffic.

Scenario	Pattern	What it tests
burst	40 s eMBB at 2 Mbps, then 35 s at 10 Mbps, repeated; URLLC steady 3 Mbps	Reaction to on/off eMBB bursts
ramp	eMBB ramps 1 → 9 → 1 Mbps; URLLC 2.5 Mbps	Tracking slow changes
sawtooth	Repeating eMBB ramps; URLLC switches between 1.5 and 3 Mbps	Changing URLLC load
adversarial	Long calm "lure" periods (URLLC 1.2), then 6–8 s "strikes" (URLLC 4.0) shorter than the EWMA time constant	Stress test against the smoothed signal's lag

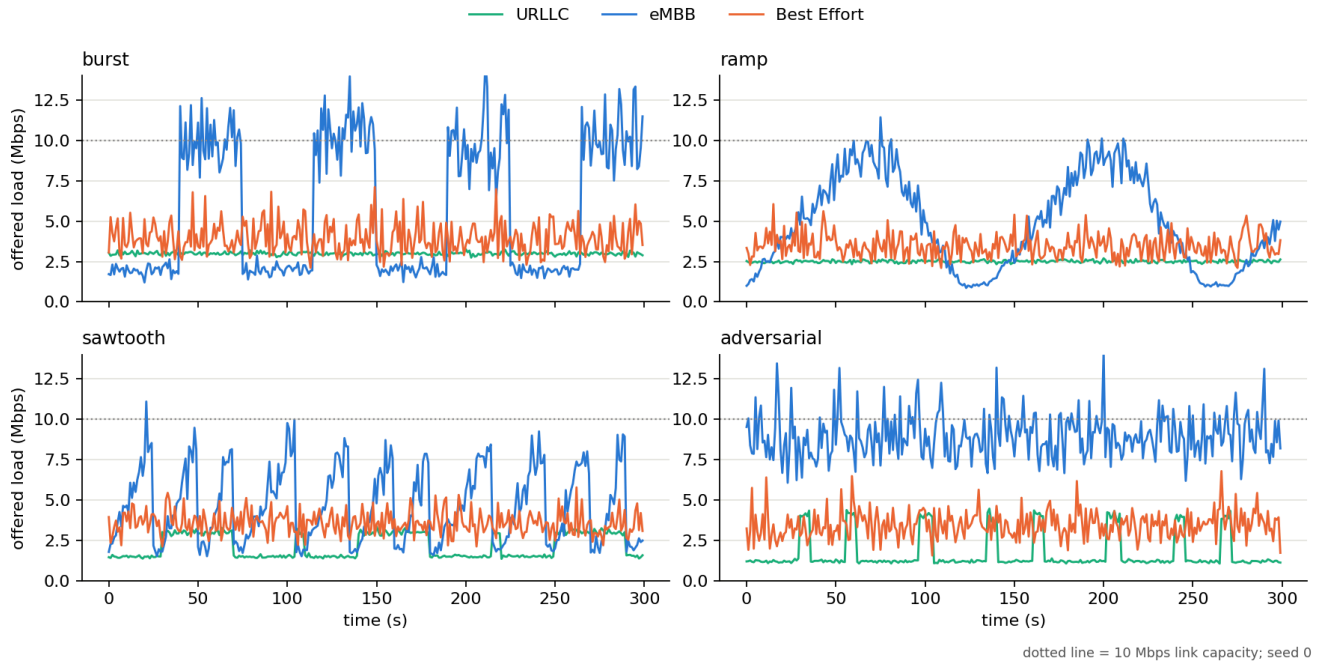


Figure 6. Offered load per slice for the four scenarios (seed 0), generated by `traffic/traces.py`.

4.3 Network interface

```
class NetworkBackend(ABC):
    def read_telemetry(self) -> TelemetrySample: ... # advance 1 s, return measurements
    def apply_allocation(self, alloc: Allocation): ... # set new rate limits
    def reset(self): ... # start a fresh run
```

`TelemetrySample` is a frozen dataclass: URLLC p50/p95 delay, per-slice throughput, per-queue drops, link utilisation and backlog. `allocation_from_level()` maps a level index to caps. URLLC is never capped, eMBB gets $level \times 10 \text{ Mbps}$, and BE is fixed at 45 %.

4.4 Queue simulator

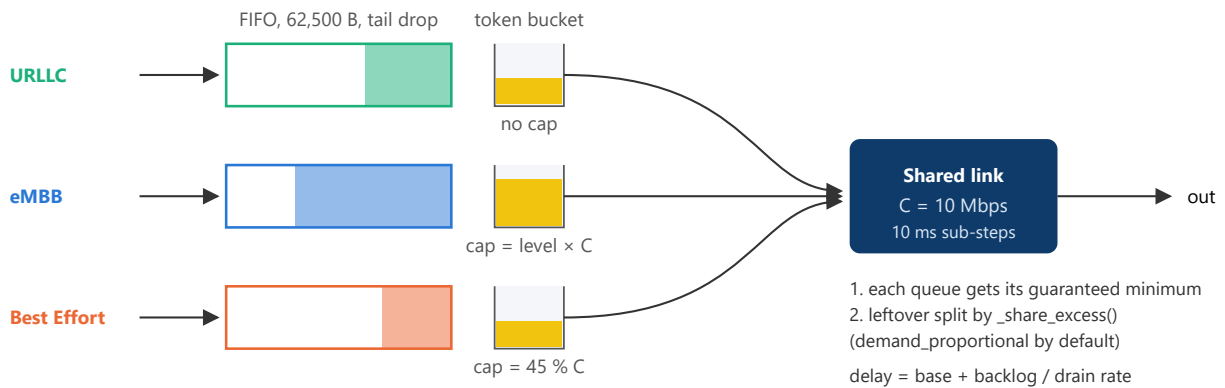


Figure 4. Simulator model: three FIFO queues, per-queue token buckets (the mechanism behind OVS HTB) and a work-conserving shared link.

Every 10 ms sub-step (`_serve_one_substep`): arrivals join each queue (overflow is tail-dropped), tokens refill, each queue requests $\min(\text{backlog}, \text{tokens})$, the link first serves guaranteed minimums (10 / 10 / 5 %) and

then splits the leftover. Every second `read_telemetry()` aggregates 100 sub-steps into throughputs, p50/p95 delay, drops, utilisation and backlog.

Engineering detail	Why it matters
Bucket size set per queue from its own cap	An early version used one fixed size, so levels 50/65/80 % behaved identically. Found and fixed.
No strict priority for URLLC	With strict priority, delay would never rise and there would be nothing to control. We use fair sharing with limits, like a real HTB switch.
Three sharing modes (<code>demand_proportional</code> , <code>equal</code> , <code>min_rate_proportional</code>)	The key modelling assumption is configurable and was later tested against real OVS (Section 10).
Fractional bytes carried forward	Drops are counted exactly rather than being rounded every sub-step.
Delay counted once, not twice	Only one direction of the link is shaped; an early plan used 2×.

4.5 Context builder (9 features)

#	Feature	Meaning	Normalised by
1	<code>bias</code>	Constant 1 (per-arm intercept)	–
2	<code>rtt_ewma_norm</code>	Smoothed URLLC delay (EWMA, $\alpha = 0.3$)	hard limit 7 ms
3	<code>rtt_p95_norm</code>	Instantaneous URLLC p95 delay	hard limit 7 ms
4	<code>urllc_util</code>	URLLC traffic (early warning <i>before</i> delay rises)	link capacity
5	<code>embb_util</code>	eMBB traffic	link capacity
6	<code>be_util</code>	Best Effort traffic	link capacity
7	<code>link_util</code>	Whole-link utilisation	–
8	<code>q0_backlog_norm</code>	URLLC bytes waiting	queue limit
9	<code>drops_norm</code>	Smoothed drops	link packet rate

All features are scaled to about 0–1 (clipped at 3), so one LinUCB `alpha` means the same thing in every dimension. The context also carries delay in **milliseconds** for the guardrail, which works in physical units.

4.6 Reward function

$r = 1.0 \times (\text{eMBB throughput} / C) + 0.3 \times (\text{BE throughput} / C) - 1.0 \times \max(0, \text{p95} - 5 \text{ ms}) / 5 \text{ ms} - 2.0 \times 1[\text{p95} > 7 \text{ ms}] - 0.5 \times (\text{drops} / \text{drop_ref})$	reward for heavy traffic served smaller reward for normal traffic sliding penalty above the target fixed penalty for breaking the hard limit penalty for dropped packets
---	--

- **p95, not the mean:** averages hide the slow packets.
- **BE has real weight (0.3):** otherwise the best policy would always hand everything to eMBB.
- **Telemetry only:** the reward never sees the policy, the action or the guardrail, so the score cannot be gamed.

- **Known defect, reported:** drops are dominated by eMBB overflow, so raising the cap *reduces* drops and `w_drop` acts as a second throughput incentive. This is why the sensitivity study sweeps it.

4.7 Policies

Every decision-maker implements `select(ctx, allowed) → int`, `update(ctx, action, reward)` and `reset(seed)`. `allowed` is the guardrail's mask.

Policy	Rule	Role
<code>static_safe</code>	Always 20 %	Never violates; wastes capacity. The floor.
<code>static_equal</code>	Always 35 % (nearest a 1/3 split)	Naive reference
<code>threshold</code>	EWMA > 5 ms → step down; < 3 ms → step up	The real competitor
<code>epsilon_greedy</code>	Context-free bandit over per-arm means	Ablation: is the context worth anything?
<code>linucb</code>	Disjoint LinUCB over the 9 features, learns during the run	The proposed method (online)
<code>*_pretrained</code>	Trained on held-out seeds, frozen, exploration off	The proposed method (converged)
<code>oracle</code>	Best phase-static schedule, knowing the trace	Reference line, not a method

5. Algorithms

5.1 LinUCB (disjoint)

Think of the five eMBB levels as five slot machines whose payout depends on the situation (the context). For each arm a , LinUCB keeps a ridge-regression model and adds an uncertainty bonus. Arms it has rarely tried in similar situations get a larger bonus, so it explores them. As data accumulates the bonus shrinks and it exploits what it has learned.

```
Initialise for each arm  $a$ :  $A_a = I$  ( $9 \times 9$ ),  $b_a = 0$  ( $9$ )
Each step, given context  $x$  and allowed mask  $M$ :
  for each arm  $a$  in  $M$ :
     $\theta_a = A_a^{-1} b_a$ 
     $UCB_a = \theta_a^T x + \alpha \cdot \sqrt{x^T A_a^{-1} x}$   $\alpha = 0.5$ 
  choose  $a^* = \operatorname{argmax} UCB_a$  (ties  $\rightarrow$  lower, safer level)
After observing reward  $r$  for the APPLIED arm  $a$ :
   $A_a^{-1} \leftarrow A_a^{-1} - (A_a^{-1} x x^T A_a^{-1}) / (1 + x^T A_a^{-1} x)$  (Sherman–Morrison)
   $b_a \leftarrow b_a + r x$ 
```

Method	Implementation
<code>scores(x, allowed)</code>	All five UCB scores at once with <code>np.einsum</code> ; disallowed arms get $-\infty$
<code>select(x, allowed)</code>	Highest score; ties go to the lower, safer level
<code>update(x, a, r)</code>	Sherman–Morrison rank-1 update of the inverse, no re-inversion. A test checks it against <code>np.linalg.inv</code> after many updates.

Cost: a few 9×9 matrix operations per decision, measured at 0.09 ms mean and under 0.25 ms p99. That is four orders of magnitude inside the 1 s control interval.

5.2 Safety guardrail

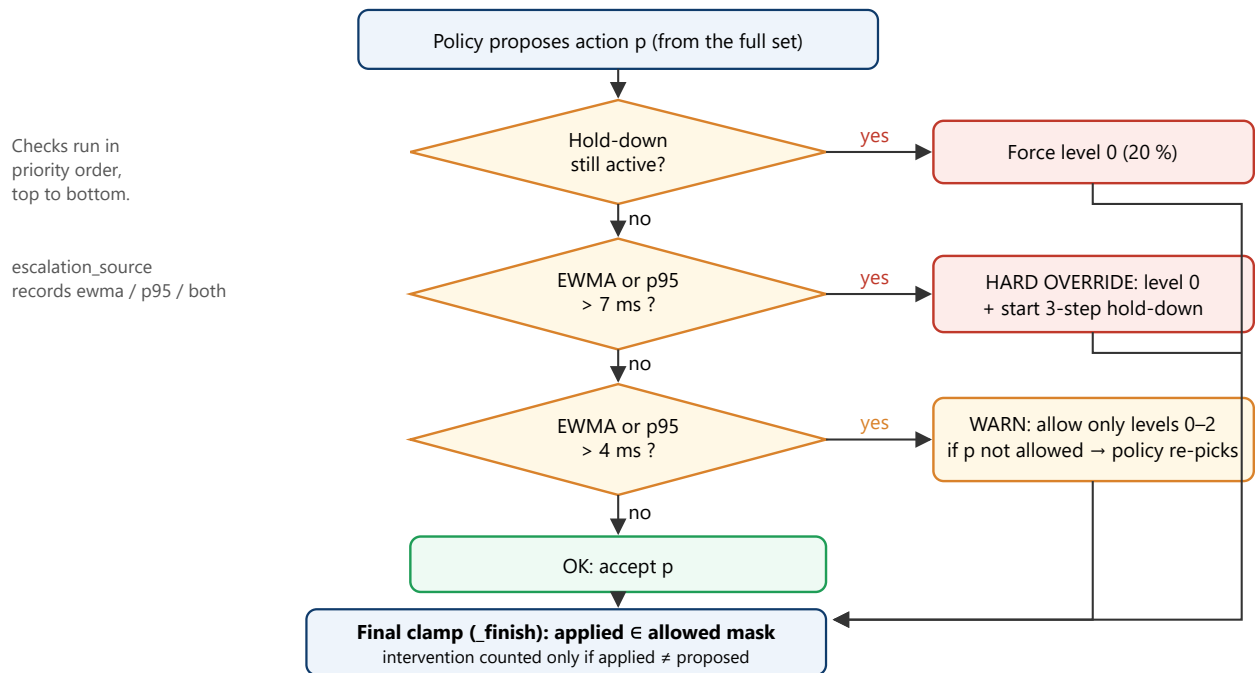


Figure 5. Guardrail decision flow. It asks the policy to re-pick among safe options rather than overruling it, and a final clamp guarantees the applied action is always allowed.

Order	State	Condition	Action
1	holddown	Cool-down still running	Force the safest level (20 %)
2	hard_override	EWMA or p95 > 7 ms	Force the safest level, start a 3-step cool-down
3	warn_mask_pass / warn_mask_reselect	EWMA or p95 > 4 ms	Allow only levels 0-2; re-pick if needed
4	ok	Normal	Accept the policy's choice

Guarantee (proved by property test): the applied action is always inside the allowed mask, whatever the policy proposes. The test uses an adversarial policy that always demands the most aggressive level, over 5,000 random cases.

Not guaranteed: zero SLA violations. The guardrail acts on the *previous* second's telemetry, so it cannot stop the first second of a spike it has not seen yet.

5.3 Leftover-capacity sharing (the assumption that mattered)

Mode	Leftover capacity is split...	Consequence for URLLC
demand_proportional (default)	in proportion to each queue's backlog	A heavy eMBB flow can crowd URLLC out, so control is needed.
equal	equally among backlogged queues	URLLC protected "for free", so no controller would be needed.
min_rate_proportional	in proportion to guaranteed minimums	URLLC largely protected.

If the real switch behaved like `equal`, the whole study would be about a modelling artifact. We flagged this in week one as the largest threat to validity, and the testbed (Section 10) was built to settle it.

6. Implementation and Testing

6.1 Technology stack

Layer	Technology
Language	Python 3 (NumPy, pandas, PyYAML, Matplotlib)
Simulation	Custom discrete-time token-bucket simulator (<code>net/sim_backend.py</code>)
Emulation	Mininet, Open vSwitch (HTB QoS), Linux <code>tc</code> , on WSL2 Ubuntu 24.04
Traffic & measurement	iperf3 3.16 (UDP), ping (ICMP RTT probes), OVS queue / drop counters
Testing	pytest with fixtures, property tests and captured iperf3 JSON
Reproducibility	blake2b seeds, config hash, <code>env.json</code> (Python, host, git commit) per run

6.2 Code size by module

Area	Main files	Lines
Agent (context, reward, guardrail, bandits, policies)	<code>agent/</code>	1,040
Network (interface, simulator, Mininet topology)	<code>net/</code>	1,359
Traffic (traces, iperf3 / ping orchestration)	<code>traffic/</code>	461
Experiments (run, suite, tune, sensitivity, 5 testbed scripts)	<code>experiments/</code>	2,385
Analysis (metrics, aggregation, plots)	<code>analysis/</code>	1,206
Configuration loader	<code>config_loader.py</code>	133
Total (excluding tests)		6,584
Tests	<code>tests/</code> (15 files)	2,917

6.3 Testing

233 automated tests (`pytest -q`) cover every module. Selected examples:

Test file	What it checks
test_sim_backend.py	Caps limit traffic; higher eMBB levels raise URLLC delay; sharing mode changes the outcome; same seed → same result
test_guardrail.py	Override, hold-down and re-pick work; applied € allowed over 5,000 random cases with a "worst-case" policy
test_reward.py	Each reward term behaves as intended
test_bandit.py , test_bandit_policy.py	LinUCB learns a textbook problem; Sherman–Morrison matches an explicit inverse
test_threshold.py	Steps up and down correctly and never double-steps
test_oracle.py	Oracle picks the best phase-static schedule
test_run_suite.py , test_suite_resume.py	Suite grid, randomised order, resume after interruption; training and evaluation seeds are disjoint
test_aggregate.py	Paired statistics and tables are recomputed from per-step logs
test_topo_parsers.py , test_generator_sweep.py	iperf3 sender vs receiver parsing (regression for a real bug), ICMP sequence-gap delivery metric
test_noise_floor.py , test_compare_sim_vs_ovs.py	Noise-floor statistics; sim-vs-OVS scoring rule

6.4 Live demonstration (excerpt)

```
python experiments/run_experiment.py --policy linucb --scenario sawtooth --seed 0 \
--set run.duration_s=60 --set run.warmup_s=0
```

step	URLLC load (Mbps)	p95 delay (ms)	proposed	applied	guardrail reason
0	1.46	2.00	0	0	ok
7	1.46	2.00	4	4	ok (explored upward)
30	3.01	2.00	4	4	ok (URLLC phase change)
35	2.76	3.70	4	2	warn_mask_reselect
36	2.86	8.19	4	4	ok (previous second was calm)
37	3.12	2.00	3	0	hard_override
38–39	2.92	2.00	3	0	holddown

Table: a real run showing exploration, re-pick, the one-second blind spot (step 36), the emergency brake and the cool-down.

7. Experimental Methodology

Principle	How it is enforced
Determinism	A run is fully determined by (policy, scenario, seed); blake2b trace seeds; a test pins this.
Paired comparison	A seed fixes the traffic bit for bit, so policies are compared on identical load . Results are the mean of per-seed differences with a 95 % t-interval.
Held-out seeds	Tuning and pre-training on seeds 100–104; evaluation on seeds 0–9. A test asserts that the two sets are disjoint.
Scale	300 s per run, first 30 s discarded, 10 seeds per cell, run order randomised from a logged seed.
Both bandit variants reported	Online (includes exploration cost, the honest deployment number) and converged (pre-trained, frozen).
Pre-registered reading rules	For every testbed stage, the rule for interpreting the result was written <i>before</i> the measurement was taken.
No cached numbers	Every table is recomputed from per-step CSV logs.

7.1 Experiment inventory

Experiment	Grid	Runs
Main evaluation suite	8 policies × 4 scenarios × 10 seeds	320
Hyperparameter tuning (LinUCB α , ϵ)	candidate values × 4 scenarios × 5 training seeds	280
Reward-weight sensitivity	$w_{sla} \{0.25 \dots 4\} \times w_{drop} \{0, 0.5, 1\} \times 5 \text{ policies} \times 4 \text{ scenarios} \times 3 \text{ seeds}$	900
Testbed stage 4 level sweep	5 levels × 3 repeats × 120 s, shuffled	15
Testbed stages 1–3, 4a, 5	topology checks, queue-size tests, noise floor, tail diagnosis, sim-vs-OVS	–

7.2 Emulation-scaled SLO

The original proposal specified a 15 / 25 ms SLA. In this setup the measured URLLC p95 ceiling across all levels is about 9 ms, so a 25 ms limit would mean zero violations for every policy and an idle guardrail. The SLA was therefore **derived from measurement**:

eMBB level	0.20	0.35	0.50	0.65	0.80
URLLC p95 (ms), burst, seed 0	2.00	3.39	5.27	7.16	9.04

Target 5 ms and hard limit 7 ms put the decision boundary between levels 0.50 and 0.65, so the choice of level actually matters.

8. Results: Simulator Policy Study

8.1 Headline: exploration decides the outcome

Paired difference in mean reward against `threshold` (10 seeds). **Bold** = 95 % interval excludes zero. Positive = better than threshold.

Policy	adversarial	burst	ramp	sawtooth
linucb (online)	-0.0728 ± 0.0182	+0.0248 ± 0.0062	-0.0217 ± 0.0029	-0.0104 ± 0.0089
linucb_pretrained	-0.0047 ± 0.0117	+0.0582 ± 0.0042	+0.0019 ± 0.0007	+0.0225 ± 0.0078
epsilon_greedy	-0.1281 ± 0.0092	+0.0082 ± 0.0199	-0.0281 ± 0.0219	-0.0201 ± 0.0235
epsilon_greedy_pretrained	-0.1287 ± 0.0114	+0.0269 ± 0.0190	+0.0019 ± 0.0007	+0.0236 ± 0.0078
oracle (reference)	+0.1241 ± 0.0115	+0.0589 ± 0.0042	+0.0019 ± 0.0007	+0.0358 ± 0.0077

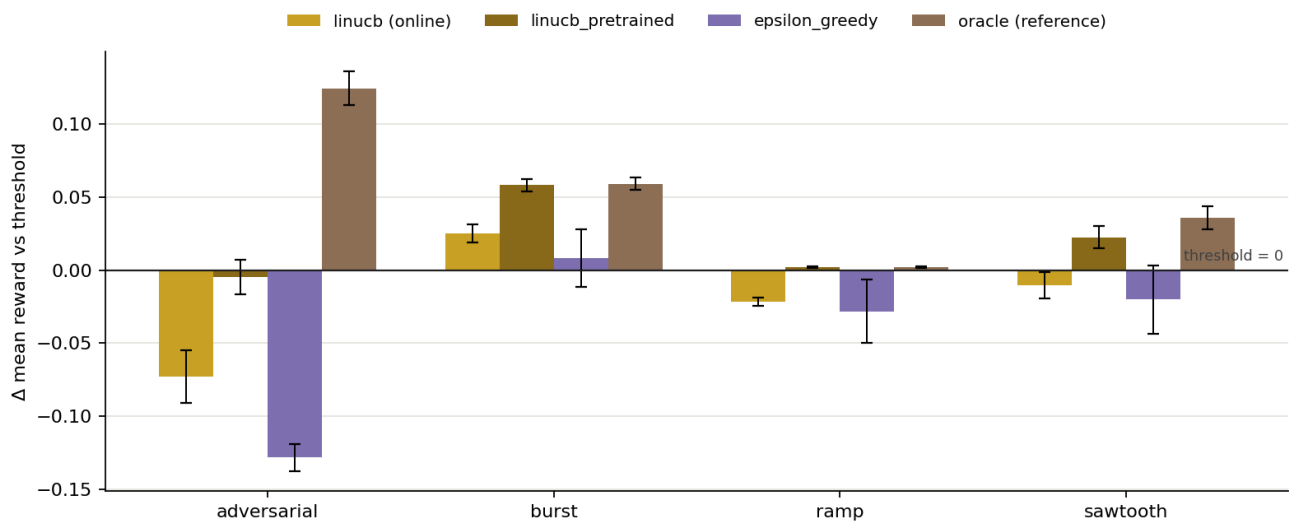


Figure 7. Paired reward difference vs `threshold` with 95 % intervals. Online LinUCB loses on 3 of 4 scenarios; pre-trained LinUCB wins on 3 of 4 and nearly matches the oracle on burst and ramp.

The pre-trained policy wins **with fewer violations**, not by trading safety for throughput. Paired difference in SLA violation rate vs threshold:

Policy	adversarial	burst	ramp	sawtooth
linucb (online)	+0.70 ± 0.40 %	-0.78 ± 0.20 %	+0.33 ± 0.08 %	0.00 ± 0.25 %
linucb_pretrained	-0.59 ± 0.42 %	-1.56 ± 0.11 %	0.00 ± 0.00 %	-0.96 ± 0.22 %

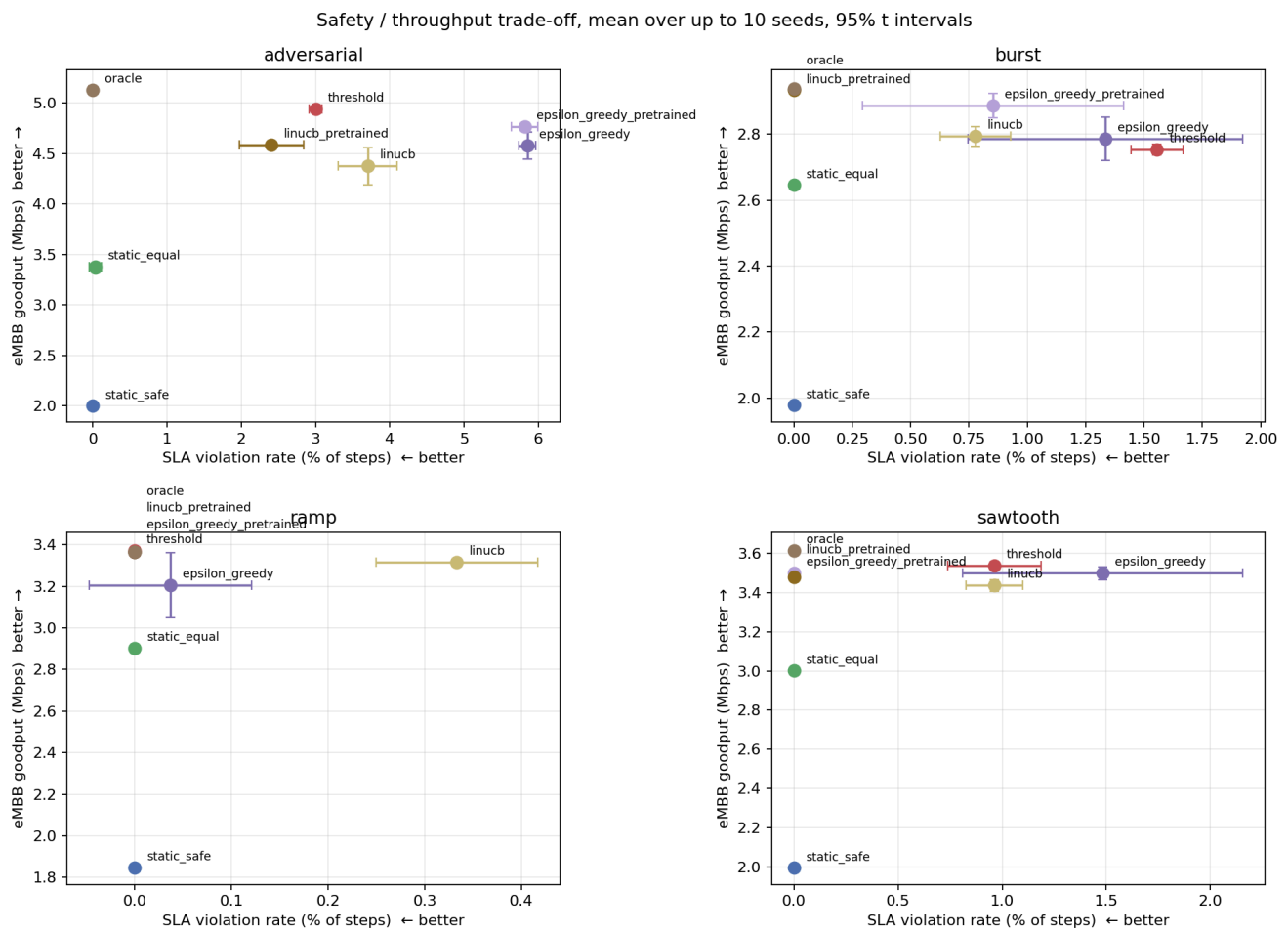
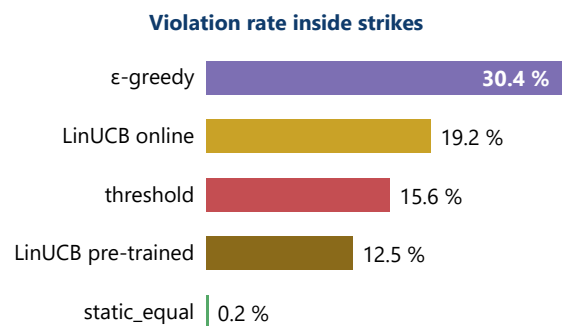


Figure 8. Safety / throughput trade-off (up and to the left is better). Pre-trained LinUCB sits near the oracle on burst, ramp and sawtooth.

8.2 The context is what the bandit contributes

On `adversarial`, all 562 violations happen inside strike phases and none in calm phases. Violation rate inside strikes:

Policy	Violations in strikes
epsilon_greedy (no context)	30.4 %
linucb (online)	19.2 %
threshold	15.6 %
linucb_pretrained	12.5 %
static_equal	0.2 %
static_safe / oracle	0.0 %



Same traffic; only the context differs

The context-free bandit is caught **more than twice as often** as the converged contextual one on identical traffic. The two differ only in whether they see the context vector.

8.3 What the guardrail does, and does not do

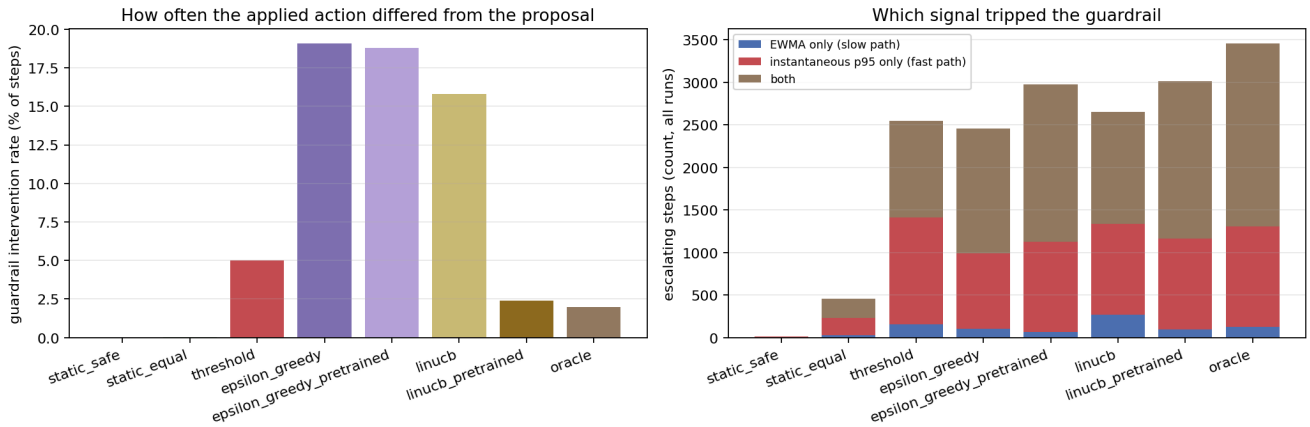


Figure 9. Left: how often the guardrail changed the policy's choice. Right: which signal escalated. The instantaneous-p95-only path (red) is a large share.

Signal that escalated (320 runs, 86,400 steps, 17,580 escalating)	Steps	Share
Smoothed EWMA only (slow path)	838	4.8 %
Instantaneous p95 only (fast path)	6,740	38.3 %
Both	10,002	56.9 %

An EWMA-only guardrail would have missed **more than a third** of all escalations. We argued for this design in week one, before any data existed, and the measurements confirmed it. The guardrail still does not remove all violations: on adversarial, `epsilon_greedy` violates on 5.85 % of steps even though the guardrail intervenes on 23.4 %.

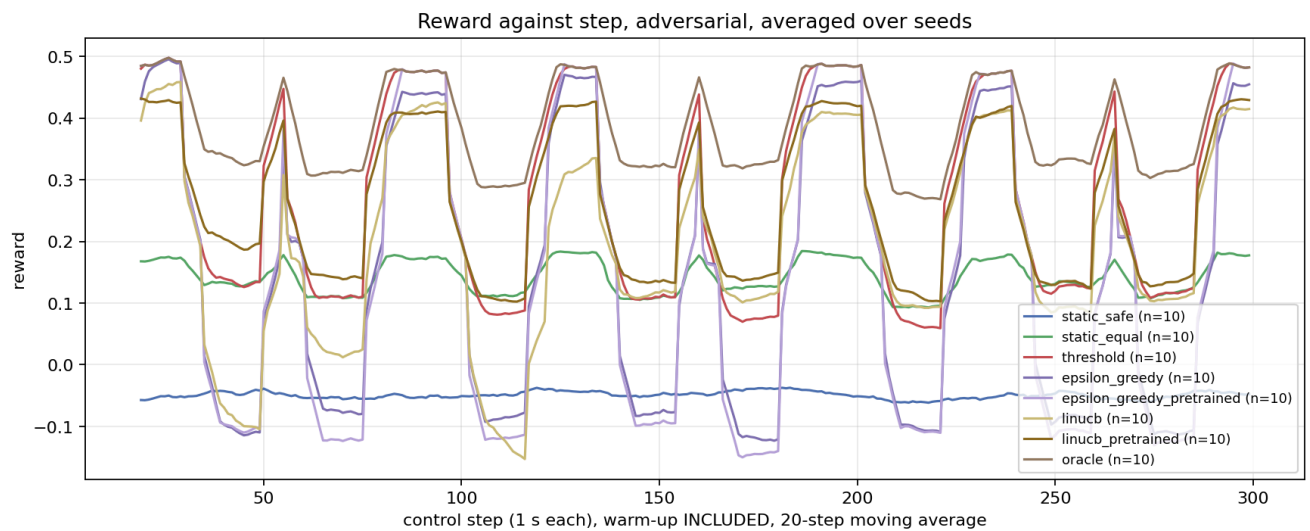


Figure 10. Reward over time on adversarial (10-seed average, 20-step moving average). The dips are the strike phases.

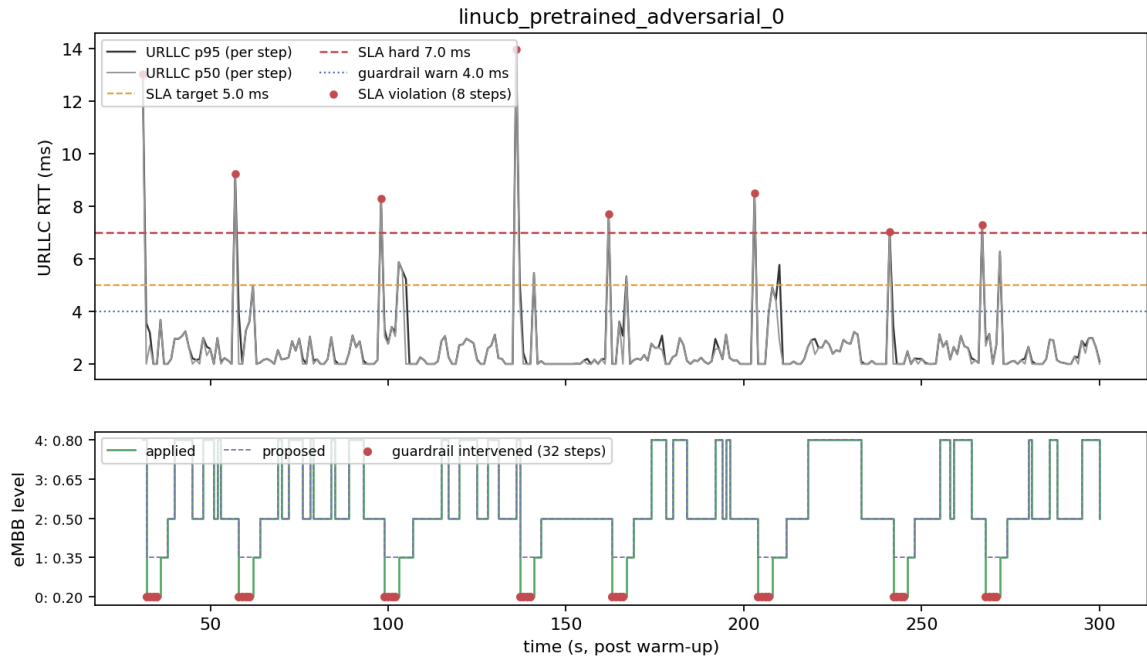


Figure 11. Pre-trained LinUCB, adversarial seed 0. Top: URLLC delay against the 4 / 5 / 7 ms lines. Bottom: proposed vs applied level; red dots mark guardrail interventions at each strike.

8.4 Regret against the best phase-static schedule

Policy	adversarial	burst	ramp	sawtooth
static_safe	+0.4290	+0.1083	+0.2194	+0.2219
static_equal	+0.2392	+0.0164	+0.0618	+0.0727
threshold	+0.1241	+0.0589	+0.0019	+0.0358
epsilon_greedy	+0.2522	+0.0507	+0.0299	+0.0559
linucb	+0.1968	+0.0342	+0.0235	+0.0462
linucb_pretrained	+0.1288	+0.0007	-0.0000	+0.0133

Lower is better; positive = worse than the oracle reference. On ramp, three policies play identical actions, so ramp cannot tell them apart.

8.5 Computational cost

Policy	Decision latency mean (ms)	p99 (ms)
Static baselines	0.021 – 0.023	0.040 – 0.049
threshold	0.035 – 0.040	0.073 – 0.115
epsilon_greedy	0.058 – 0.066	0.117 – 0.163
linucb	0.091 – 0.098	0.192 – 0.233

8.6 Reward-weight sensitivity (900 runs)

`linucb_pretrained` ranks first in **all 15** ($w_{sla} \times w_{drop}$) cells, and the win is separated from zero in 13. Both exceptions are at the weakest SLA weight. The runner-up changes with the weights. Only the learned policies

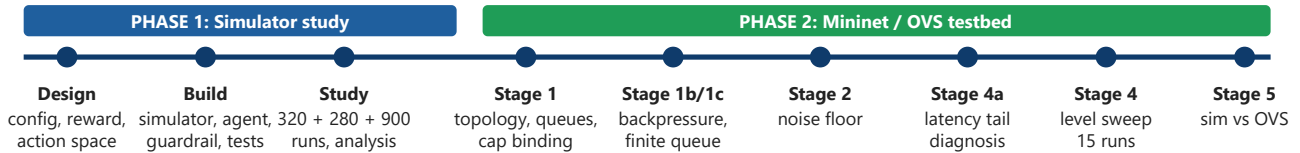
respond to the weights at all:

SLA violation rate	w_sla 0.25	0.50	1.00	2.00	4.00
linucb_pretrained	0.70 %	0.74 %	0.64 %	0.66 %	0.37 %
linucb	1.51 %	1.46 %	1.35 %	1.17 %	0.88 %
threshold	1.54 %	1.54 %	1.54 %	1.54 %	1.54 %
static_equal	0.00 %	0.00 %	0.00 %	0.00 %	0.00 %

At $w_{sla} = 0.25$, pre-trained LinUCB delivers **3.634 Mbps at 0.70 % violations**, against threshold's **3.655 Mbps at 1.54 %**: the same throughput to within 0.6 %, with less than half the violations.

9. Real Testbed: Mininet and Open vSwitch

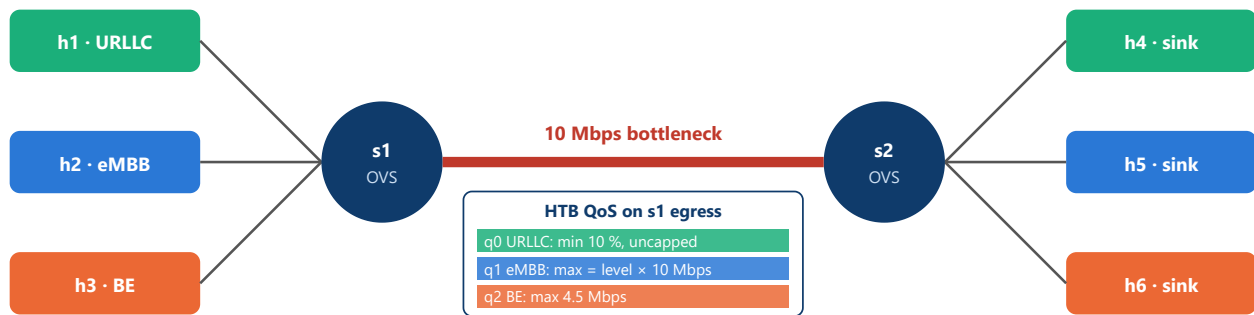
The whole simulator study rested on one unchecked assumption: how leftover capacity is shared (Section 5.3). Phase 2 built a real testbed to check it. The testbed was built in stages, each one gated on the previous stage producing output, and each with its reading rule written down before the measurement.



Every stage had a pre-recorded decision table, including outcomes that would have invalidated the simulator study.

Figure 12. Project timeline: the simulator phase and the gated testbed stages.

9.1 Topology



Port-based OpenFlow rules classify traffic into queues · iperf3 UDP load · ping as the URLLC latency probe

Figure 13. Mininet topology $h1, h2, h3 - s1 = s2 - h4, h5, h6$, with all queueing on s1's egress. Built by `net/topology/slice_topo.py` on WSL2 Ubuntu 24.04.

9.2 Stage-by-stage results

Stage	Question	What was found
1	Does the topology work and do caps bind?	Connectivity, queues and tc classes all passed. Found a bug in our own check: <code>iperf 3.16's end.sum</code> is the <i>sender's</i> rate (4.002 Mbps) and not the receiver's (0.976). After the fix, the cap binds exactly: 3.398 Mbps predicted, 3.399 measured .
1b	Does excess traffic get dropped?	No. The sender was blocked (backpressure), because Mininet runs sender and switch in one kernel. This is an emulation artifact.
1c	Does a finite queue restore drops?	Yes. All three predictions held (table below). Actuation latency was 21–62 ms, more than 15× inside the control interval.
2	What is the idle latency floor?	p50 0.093 ms, p95 0.133, p99 0.166 (1,182 samples, 0 loss). The simulator's 2.0 ms base is about 20× larger.
4a	Where does the latency tail come from?	Host (laptop / Hyper-V) scheduling, not the network. Real-time-priority ping did not remove it. Median became the primary URLLC metric . The delivery metric was also fixed to use ICMP sequence gaps.
4	Does slicing work on real OVS?	Yes (table in 9.3).
5	Which simulator mode matches the switch?	demand_proportional , the mode the whole policy study used (Section 10).

Stage 1c: queue size, predicted before measuring

Queue limit	Predicted	Result
OVS default, 1,000 packets	blocks the sender	blocks, backlog 134,106 B
62,500 B (the simulator's value)	drops instead	drops , backlog 62,006 B = exactly 43 frames
500,000 B (control)	blocks again	blocks, backlog 134,106 B again

9.3 Stage 4: level sweep on real Open vSwitch

Constant load (URLLC 3, eMBB 10, BE 4 Mbps), 3 repeats × 120 s per level, shuffled order. Calibration passed and all 15 of 15 runs were valid. Mean ± 95 % interval:

eMBB level	eMBB (Mbps)	BE (Mbps)	URLLC median RTT	URLLC drops
0.20	1.995 ± 0.009	3.997 ± 0.015	0.10 ± 0.03 ms	0
0.35	3.270 ± 0.030	3.638 ± 0.010	2.63 ± 0.21 ms	0
0.50	4.082 ± 0.083	2.812 ± 0.020	3.65 ± 0.27 ms	0
0.65	4.424 ± 0.015	2.465 ± 0.022	4.44 ± 0.44 ms	0
0.80	5.087 ± 0.018	1.796 ± 0.013	7.67 ± 0.24 ms	0

- Raising the eMBB cap moves capacity from Best Effort to eMBB at every step.
- URLLC **throughput** is fully protected: zero drops and 100 % probe delivery in all 15 runs.
- URLLC **latency** pays the price: about **77x** from the lowest level to the highest, and adjacent levels' intervals do not overlap.
- Every packet is accounted for: delivered + dropped = 99.6–100.0 % of offered in every run.

This settles the project's largest open question: the control problem exists, by measurement.

10. Simulator vs Real Switch

The simulator was run under matched conditions (same load, no jitter, the measured 0.093 ms base latency) in all three sharing modes. It was scored by a rule recorded in advance: a normalised error of at most 0.25 counts as "tracking".

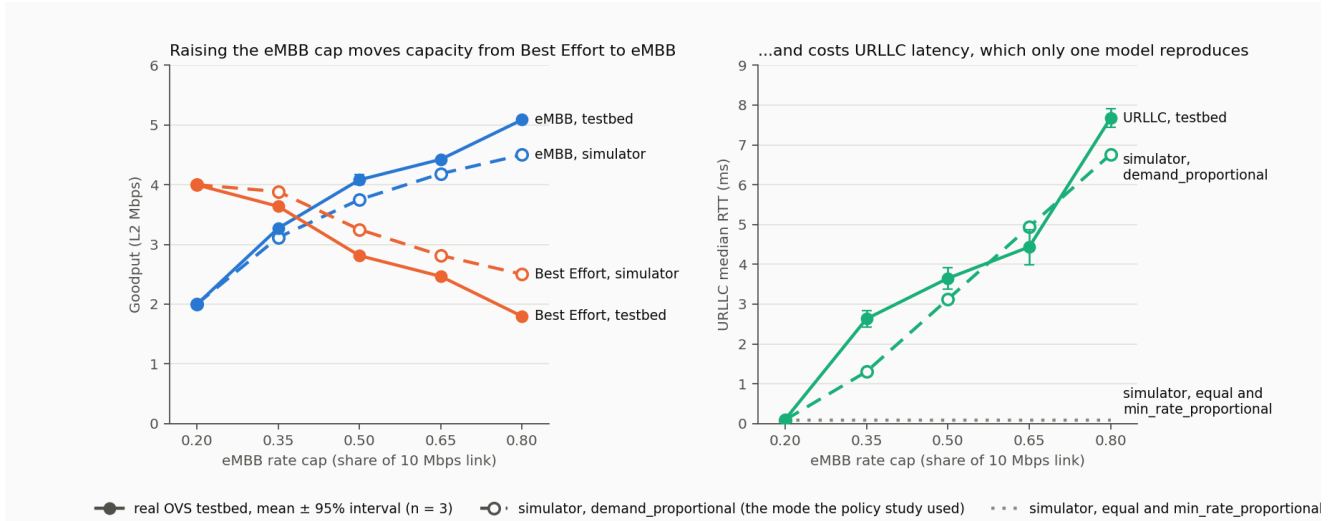


Figure 14. Real OVS (solid, mean $\pm$ 95 % CI) vs simulator (dashed = *demand_proportional*, dotted = other modes). Only *demand_proportional* reproduces the rise in URLLC latency.

Mode	eMBB goodput	BE goodput	URLLC median	Mean error	Verdict
demand_proportional	0.085	0.158	0.087	0.110	TRACKS
equal	0.167	0.256	0.476	0.300	does not track
min_rate_proportional	0.096	0.117	0.476	0.230	goodput only; latency wrong

Result: **TRACKS demand_proportional**. The other two modes predict 0.09 ms URLLC latency at every level, and the switch contradicts that by about 85× at the top level.

Honest qualifications. The verdict is sensitive to the threshold: at a threshold of 0.15 or stricter it becomes "partial". The simulator is also systematically biased. At congested levels it **underestimates eMBB goodput by 5–12 %** and **overestimates BE by 7–39 %**. The real switch favours eMBB over BE more than the model does. In short, the simulator gets the direction of the effect right but its size wrong.

11. Discussion and Key Findings

#	Finding	Evidence
1	Slicing works on a real programmable switch.	Caps bind to within 0.001 Mbps; URLLC keeps full throughput; the eMBB cap trades against URLLC latency (Stage 4).
2	The control problem is real, not a modelling artifact.	Two of the three plausible models predicted free protection; the switch contradicts them (Stage 5).
3	Exploration, not learning, is the deciding cost.	Online LinUCB loses on 3/4 scenarios; pre-trained LinUCB wins on 3/4 with fewer violations (Section 8.1).
4	The context is what the bandit contributes.	The context-free bandit is caught 30.4 % vs 12.5 % in strikes (Section 8.2).
5	The guardrail bounds actions, not violations.	Property test over 5,000 cases, and 5.85 % violations despite 23.4 % intervention (Section 8.3).
6	The fast signal is load-bearing.	38.3 % of escalations came from instantaneous p95 alone.
7	LinUCB is lightweight.	< 0.25 ms p99 per decision against a 1,000 ms interval.
8	The simulator gets the direction right and the size wrong.	Mean error 0.110, with eMBB / BE biases of 5–39 %.

Practical recommendation

If a deployment can **train offline** before going live, a contextual bandit behind a deterministic guardrail is worth using: it beats a hand-tuned rule and has fewer SLA violations. If it must **learn in production**, a well-tuned threshold rule is the better engineering choice at this scale. Any system that claims safety from a guardrail should say whether it bounds *actions* or *violations*.

Mistakes found and corrected

Problem	How it was caught	Fix
Cap check read iperf3's sender rate as the receiver's	Controlled 1 Mbit test: 4.002 vs 0.976 Mbps	Parse sender and receiver separately; superseded result kept
"Ping delivered" read 90–98 % while ping reported 0 % loss	Compared against ping's own summary	Count ICMP sequence-number gaps
Stage 1b/1c shaper rate read about 10 % low	Rate divided by wall time that included iperf3 setup and the drain tail	Use only counter samples inside the flow window
Simulator levels 50/65/80 % behaved identically	Level sweep in the simulator	Bucket size per queue

12. Limitations

1. **One operating point, constant load.** The testbed comparison uses a single offered load with no jitter. It does not show agreement under the fluctuating traffic the policies were evaluated on.
2. **No policy has run on the real switch yet.** Every policy result is a simulator result.
3. **Tail latency on this hardware is noisy.** Host jitter puts the uncongested p95 at 5–12 ms, against a 7 ms limit.
4. **Rounds are not independent.** Backlog and autocorrelated load carry state from one round to the next, which a bandit does not assume.
5. **Ten seeds of one simulator is not ten samples of reality.**
6. **The scenarios are hand-written.** `adversarial` is a stress test, not a traffic model.
7. **The reward weights decide the runner-up,** though not the winner.
8. **OVS-native QoS, not an OpenFlow controller.** The data plane is real and programmable, but allocations are pushed through the OVS management interface.

13. Conclusion and Future Work

SafeSlice shows that a contextual bandit combined with a deterministic guardrail is a workable, lightweight design for SLA-preserving dynamic slicing. We state three qualifications, each supported by measurement rather than assumed. **Exploration is the deciding cost:** online learning loses to a tuned rule, while offline pre-training wins. **Context is the contribution:** without it the bandit is caught twice as often. **The guardrail's guarantee is narrow and provable:** it bounds the applied action, not the violation rate.

The Mininet / Open vSwitch testbed confirmed that network slicing works on a real switch, that the latency trade-off the controller manages really exists, and that the simulator's sharing model is the correct one of the three. It also showed the simulator's systematic biases.

Future work

Work item	Purpose
net/ovs_cli_backend.py	Implement <code>NetworkBackend</code> for OVS so the existing policies run live, with no agent changes.
Live policy loop on the testbed	Evaluate LinUCB + guardrail on real traffic; needs a fix for host-level tail latency first.
Validate under fluctuating load	Extend the sim-vs-OVS comparison beyond one constant operating point.
Calibrate simulator biases	Correct the eMBB / BE goodput bias found in Stage 5.
OpenFlow controller path	Push allocations through an SDN controller (for example ONOS or OpenDaylight).
Safer online exploration	Warm-start or conservative bandits to reduce the cost of exploration.

References

1. Koo et al., "Deep Reinforcement Learning for Network Slicing," 2019.
2. Wei et al., "Dynamic Network Slice Reconfiguration using Deep Reinforcement Learning," 2020.
3. Dayot et al., "Deep Contextual Bandit-Based Slice Provisioning," 2022.
4. "Safe and Fast Reinforcement Learning for Network Slicing Resource Allocation," 2023.
5. Xie et al., "DRL-Based Resource Allocation for Network Slicing," 2022.
6. L. Li, W. Chu, J. Langford, R. E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," WWW 2010.
7. Mininet: An Instant Virtual Network on your Laptop, *mininet.org*.
8. Open vSwitch documentation: QoS and HTB queues, *docs.openvswitch.org*.
9. iperf3 network performance tool, *software.es.net/iperf*.

Appendix A. Reproducing the results

Simulator (any machine, no root):

```
pip install -r requirements.txt && pytest -q      # 233 tests
python experiments/run_suite.py                  # 320 runs
python -m analysis.aggregate                     # every simulator table
python -m analysis.plots                         # every simulator figure
```

Testbed (Linux with root; setup in `docs/TESTBED_SETUP.md`):

```
python3 net/topology/slice_topo.py --check      # topology, queues, cap binding
python3 experiments/measure_noise_floor.py      # idle latency floor
python3 experiments/sweep_levels_ovs.py        # 15 runs, about 32 min
python experiments/compare_sim_vs_ovs.py       # simulator vs switch
python -m analysis.plot_testbed                 # Figure 14
```

Appendix B. Repository map

Path	Contents
config/	Default settings and the four scenario files
net/backend.py	The interface everything is written against
net/sim_backend.py	The queue simulator
net/topology/slice_topo.py	Mininet topology, OVS QoS, testbed checks
traffic/	Trace generation; iperf3 and ping orchestration
agent/	Context, reward, guardrail, bandits, policies
experiments/	Single run, suite, tuning, sensitivity, 5 testbed experiments
analysis/	Metrics, aggregation, figures
results/summary/	All summary CSV / JSON results and figures
docs/	Plan, testbed plan, design decisions, experiments, overview
tests/	233 automated tests